

GIGW 3.0 ACCESSIBILITY AUDIT TOOLKIT

User Manual

A plain-language guide to the Web Crawler, Media Crawler, Alt-Text Checker, and Colour Contrast Checker

Prepared for the Website Testing & Evaluation Team
Version 1.0

Contents

Contents	2
1. What This Toolkit Does	4
2. How the Four Tools Fit Together.....	4
2.1 The Pipeline at a Glance	4
3. Before You Start	5
4. One-Time Setup.....	5
4.1 Check that Python is installed.....	5
4.2 Install the helper packages	5
4.3 Put the scripts in one folder	5
5. Tool 1 — The Web Crawler (crawler.py)	7
5.1 What it does	7
5.2 How to run it	7
5.3 Useful options.....	7
5.4 What you get	7
6. Tool 2 — The Media Crawler (media_crawler.py).....	9
6.1 What it does	9
6.2 How to run it	9
6.3 Useful options.....	9
6.4 What you get	9
7. Tool 3 — The Alt-Text Checker (alt_text.py).....	11
7.1 What it does	11
7.2 How to run it	11
7.3 Useful options.....	11
7.4 What you get	11
8. Tool 4 — The Colour Contrast Checker (contrast_checker.py).....	13
8.1 What it does	13
8.2 How to run it	13
8.3 Useful options.....	13
8.4 What you get	14
9. A Complete Walkthrough	15
10. Reading and Using the Reports	15
11. Important Limitations to Know	15
12. Troubleshooting	17
13. Quick Command Reference	18
14. Glossary.....	18

1. What This Toolkit Does

This toolkit is a set of four small programs (“scripts”) that work together to help test government websites for accessibility, in line with GIGW 3.0 (Guidelines for Indian Government Websites) and the underlying WCAG (Web Content Accessibility Guidelines) standards.

Without this toolkit, a tester has to open every single page of a website by hand and check things like whether images have descriptions, whether videos or audio clips are present, and whether the text is easy to read against its background colour. On a large government website with hundreds of pages, that is slow and it is easy to accidentally miss something.

This toolkit automates the repetitive, mechanical parts of that work. It can:

- Find every page on a website automatically, the way a visitor clicking every link would.
- Build a complete inventory of every image, video, audio clip, and embedded item on the site.
- Check whether every image has a proper text description for screen-reader users.
- Check whether the text colours on every page are easy enough to read against their backgrounds.

The output of each tool is a spreadsheet (a CSV file, which opens directly in Excel) that a tester can sort, filter, and use as the working basis for an audit report.

Important: These tools find likely problems automatically, but they cannot replace human judgement. Every flagged item (and a sample of the items that passed) should still be reviewed by a tester before it is reported as a confirmed finding. Section 12 explains specifically what each tool can and cannot see.

2. How the Four Tools Fit Together

Think of the toolkit as a small assembly line. The first tool builds a master list of every page on the website. The other three tools each read that list and use it to know which pages to visit.

2.1 The Pipeline at a Glance

Step	Tool	What it produces
1	Web Crawler (crawler.py)	A list of every page on the site — the master crawl file (crawl.csv)
2a	Media Crawler (media_crawler.py)	An inventory of every image, video, audio clip, and embed on the site
2b	Alt-Text Checker (alt_text.py)	A pass/fail report on image descriptions (WCAG 1.1.1)
2c	Colour Contrast Checker (contrast_checker.py)	A pass/fail report on text-vs-background contrast (WCAG 1.4.3 / 1.4.6)

Steps 2a, 2b, and 2c do not depend on one another — they only depend on Step 1’s output. That means once the Web Crawler has finished, you can run the Media Crawler, the Alt-Text

Checker, and the Colour Contrast Checker in any order, one after another, on the same crawl file.

Note: Every tool after the Web Crawler is pointed at the crawl CSV produced in Step 1. If you re-crawl a site and get a new CSV, re-run the other tools against the new file so the reports stay in sync with the current state of the website.

3. Before You Start

To use this toolkit you will need:

- A computer with Python installed (version 3.9 or newer is recommended).
- An active internet connection — the tools fetch live pages from the website being tested, the same way a web browser does.
- The starting web address (URL) of the site you want to test, e.g. <https://example.gov.in>
- Basic comfort opening a “Terminal” (Mac/Linux) or “Command Prompt” / “PowerShell” (Windows) window and typing a line of text. This manual shows the exact text to type at every step — no programming knowledge is required.
- The four script files — `crawler.py`, `media_crawler.py`, `alt_text.py`, and `contrast_checker.py` — saved together in one folder on your computer.

Setup takes about ten minutes the first time, and only needs to be done once per computer.

4. One-Time Setup

The scripts rely on a handful of small, free helper packages. Installing them is a one-time step — you will not need to repeat it for future audits on the same computer.

4.1 Check that Python is installed

Open a Terminal / Command Prompt window and type:

```
python --version
```

If a version number (3.9 or higher) appears, you’re ready. If you see an error, install Python from python.org first, then come back to this step.

4.2 Install the helper packages

In the same window, type the following command and press Enter:

```
pip install requests beautifulsoup4 pandas openpyxl tinycss2
```

This downloads everything all four tools need: a tool to fetch web pages, a tool to read the HTML on those pages, a tool to organise data into tables, a tool to write Excel files, and a tool to help read colour styling.

4.3 Put the scripts in one folder

Save `crawler.py`, `media_crawler.py`, `alt_text.py`, and `contrast_checker.py` into a single folder, for example a folder named `GIGW-Audit-Tools`. Open your Terminal / Command Prompt and navigate into that folder before running any of the commands in this manual, for example:

Windows example

```
cd Documents\GIGW-Audit-Tools
```

Mac/Linux example

```
cd ~/Documents/GIGW-Audit-Tools
```

5. Tool 1 — The Web Crawler (crawler.py)

5.1 What it does

Picture a robot that starts at a website's homepage, opens every link it finds, then opens every link on each of those pages, and keeps going until it has visited every page on the site (or as many as you tell it to). As it goes, it writes down each page's web address, its title, whether it loaded successfully, which page led to it, and how many "clicks" deep it sits in the site.

This is always the first tool to run. The list of pages it produces is the foundation that the other three tools build on — they each need to know which pages exist before they can check anything on them.

The crawler stays within the same website domain — it will not wander off to link to other organisations' websites — and politely waits a short pause between page visits so it doesn't overload the server being tested.

5.2 How to run it

Basic usage — simply provide the website's starting address:

```
python crawler.py --url https://example.gov.in
```

If you run the script without `--url`, it will simply ask you to type the address when it starts.

As it runs, the Terminal window shows live progress: each page address as it's visited, its result, and how many new links it found. For a large site this can take a while — let it run to completion.

5.3 Useful options

Option	What it does	Example
<code>--url</code>	The website address to start crawling from. Asked for interactively if left out.	<code>--url https://example.gov.in</code>
<code>--depth</code>	How many "clicks" deep to go. Leave it out for an exhaustive, unlimited crawl (the default).	<code>--depth 3</code>
<code>--delay</code>	Seconds to pause between page visits (default 1 second). Increase this for slower or more sensitive servers.	<code>--delay 2</code>
<code>--output</code>	The name/location of the report file. A sensible default name is generated automatically if omitted.	<code>--output crawl.csv</code>
<code>--format</code>	csv (default, works with the other three tools) or xlsx for a polished, colour-coded Excel workbook.	<code>--format xlsx</code>

5.4 What you get

By default you get a CSV file (named after the website, e.g. `example.gov.in_crawl.csv`) with one row per page found. The columns are:

Column	Meaning
URL	The exact web address of the page.

Column	Meaning
Page Title	The title shown in the browser tab for that page.
HTTP Status	The page's response code. 200 means it loaded successfully; codes like 404 or 500 indicate a broken or missing page.
Found On (Parent URL)	The page that contained the link leading here — useful for tracing how a broken page is reached.
Depth	How many clicks away from the homepage this page sits.
Discovered At	The time of day the page was visited during the crawl.
Notes	Extra detail for problem pages, e.g. "HTTP 404", "Request timed out", or "Non-HTML content".

Note: Choosing `--format xlsx` instead gives you a formatted Excel workbook with three tabs — Crawl Results, a Summary (totals and a breakdown by depth and status code), and a dedicated Errors & Skipped tab — which is convenient for a quick management-level overview. The other three tools, however, expect a CSV, so for the audit pipeline keep the default CSV output.

Important: The crawl report is only written once the crawl finishes completely. If you stop the crawler partway through (for example with Ctrl+C) on a very large site, no file will be saved. If a site is extremely large, use `--depth` to set a sensible limit rather than interrupting the crawl midway.

6. Tool 2 — The Media Crawler (media_crawler.py)

6.1 What it does

Once the Web Crawler has the full list of pages, the Media Crawler revisits every one of those pages and looks specifically for anything that isn't plain text: photographs, illustrations, videos, audio clips, embedded players (such as an embedded YouTube video), and inline graphics (SVGs). It also picks up images that are set as CSS background images.

The result is a complete inventory of every media item on the site — something testers need when checking rules around captions, transcripts, and other media-related accessibility requirements, and simply to know the scale of media content a site contains.

For each media item found, it also checks (optionally) whether the file itself still loads, which helps surface broken images or dead video links.

6.2 How to run it

Point it at the crawl CSV produced by the Web Crawler:

```
python media_crawler.py --input crawl.csv
```

If you leave out `--input`, the script will ask you to type the path to the crawl CSV.

6.3 Useful options

Option	What it does	Example
<code>--input</code>	Path to the crawl CSV from the Web Crawler.	<code>--input crawl.csv</code>
<code>--output</code>	Base name for the report files.	<code>--output media_report</code>
<code>--format</code>	csv (default, produces two files) or xlsx (one combined workbook).	<code>--format xlsx</code>
<code>--delay</code>	Seconds to pause between page visits (default 0.5).	<code>--delay 1</code>
<code>--timeout</code>	How long to wait for a slow page before giving up, in seconds (default 10).	<code>--timeout 15</code>
<code>--no-verify</code>	Skip checking whether each media file's own link still works. Faster, but no status information.	<code>--no-verify</code>

6.4 What you get

By default, two CSV files are produced:

- A detail file — one row for every individual media item found across the whole site.
- A summary file — one row per page, with counts of how many images, videos, audio clips, iframes, and so on appear on it.

The detail file's columns are:

Column	Meaning
<code>parent_url</code>	The page the media item was found on.
<code>media_type</code>	image, video, audio, iframe, embed, object, svg (inline), or image (CSS bg).

Column	Meaning
media_url	The address of the media file itself (or “(inline — no external URL)” for inline graphics).
tag	The HTML element that contained the media reference, e.g. or <video>.
alt_title	Any alt text or title attached to the item, if present.
http_status	Whether the media file itself loads (e.g. 200 = OK, 404 = broken). Shows N/A if verification was skipped.

Note: Choosing --format xlsx gives a three-tab Excel workbook: Media Detail (every item), Page Summary (counts per page), and Stats (site-wide totals and the five pages with the most media — a quick way to spot media-heavy pages worth a closer manual look).

7. Tool 3 — The Alt-Text Checker (alt_text.py)

7.1 What it does

This tool checks every page against WCAG Success Criterion 1.1.1 (Non-Text Content) — one of the most commonly tested accessibility rules. In plain terms: every meaningful image on a webpage should have a short written description, called “alt text”, so that someone using a screen reader, or someone whose images fail to load, still understands what the image shows or what a clickable image button does.

The tool revisits every page from the crawl list and inspects every image, every clickable image button, every image-map area, and every inline graphic (SVG) on the page, checking each against this rule.

Examples of what it flags as a failure:

- An image with no description at all.
- An image whose description is just a meaningless placeholder, like “image”, “photo”, “icon”, or “click here”.
- A clickable image button (e.g. a search icon used as a submit button) with no label.
- A clickable area within an image map that has no description.
- An inline graphic (SVG) with no accessible name, where one is needed.

It correctly does not flag purely decorative images that have been deliberately marked as decorative (for example, with an empty alt="" attribute or role="presentation") — those are allowed under the rule and don’t need a description.

7.2 How to run it

Provide the crawl CSV as input, and choose a name for the report it should produce:

```
python alt_text.py crawl.csv wcag_report.csv
```

7.3 Useful options

Option	What it does	Example
--delay	Seconds to pause between page visits (default 0.5).	--delay 1
--limit	Only check the first N pages — handy for a quick test run before checking the whole site.	--limit 20

7.4 What you get

A CSV report with one row per page. The columns added to the original crawl data are:

Column	Meaning
Result	PASS (no issues found), FAIL (one or more issues found), ERROR (the page could not be checked), or SKIP (the page was broken or excluded).
Total Images	How many images / image-like elements were found and checked on the page.
Pass Count	How many of those passed the check.

Column	Meaning
Fail Count	How many failed.
Issues	A plain-language list of exactly what failed and where, e.g. which image and what was wrong with it.

Note: When you run the tool, it also prints a running tally as it works, and finishes with a summary count of PASS / FAIL / ERROR / SKIP totals for the whole site — useful as a quick health check before opening the spreadsheet.

8. Tool 4 — The Colour Contrast Checker (contrast_checker.py)

8.1 What it does

This tool checks every page against WCAG Success Criteria 1.4.3 and 1.4.6 (Contrast — Minimum, and Contrast — Enhanced). In plain terms: text needs to stand out clearly enough from its background colour so that people with low vision, colour blindness, or who are simply reading on a low-quality screen or in bright sunlight, can still read it comfortably.

The rule is based on a mathematical measure called a “contrast ratio” — essentially, how different the lightness of the text colour is from the lightness of its background. A ratio of 1:1 means the colours are identical (unreadable); a ratio of 21:1 is the maximum possible (pure black text on a pure white background).

AA versus AAA

AA is the everyday, baseline standard most websites are expected to meet. AAA is the stricter “gold standard” version, sometimes required only for particularly important pages. The tool defaults to AA but can be switched to AAA with a single option.

Conformance level	Normal text needs at least...	Large / bold text needs at least...
AA (default, everyday standard)	4.5 : 1	3.0 : 1
AAA (stricter standard)	7.0 : 1	4.5 : 1

“Large text” is text that is roughly 24px (18pt) or bigger when regular weight, or roughly 18.67px (14pt) or bigger when bold. Large or bold text gets a slightly lower bar because it is naturally easier to read even with somewhat less contrast.

8.2 How to run it

Basic usage, checking against the default AA standard:

```
python contrast_checker.py crawl.csv
```

To check against the stricter AAA standard instead:

```
python contrast_checker.py crawl.csv --level AAA
```

8.3 Useful options

Option	What it does	Example
--level	AA (default) or AAA — which WCAG standard to test against.	--level AAA
--custom-threshold	Override the WCAG standard with one single ratio applied to all text — useful for testing against an internal house standard.	--custom-threshold 5.0
--output	Folder to save the report files into (default: the current folder).	--output results/
--workers	How many pages to check at the same time. Higher is faster but puts more load on the website and your computer; 3 is a sensible default, 8 a reasonable maximum.	--workers 4

Option	What it does	Example
--timeout	How long to wait for a slow page before giving up, in seconds (default 20).	--timeout 30
--no-verify	Turn off SSL certificate checking — useful for internal/staging sites with self-signed certificates.	--no-verify
--sample	Only check the first N pages — handy for a quick spot-check before running the whole site.	--sample 10
--delay	A polite pause between requests on each worker (default 0.5 seconds).	--delay 1

8.4 What you get

Two timestamped CSV files are produced — `contrast_detail_<date-time>.csv` and `contrast_summary_<date-time>.csv`.

The detail file has one row per piece of text checked:

Column	Meaning
url	The page the text was found on.
element	The type of HTML element the text is in, e.g. p, h1, button.
text_sample	A short snippet of the actual text, so you can locate it on the page.
fg_colour / bg_colour	The text colour and background colour that were compared, shown as hex codes (e.g. #000000).
contrast_ratio	The calculated ratio between those two colours.
required_ratio	The minimum ratio this particular piece of text needed to meet, based on its size and the chosen WCAG level.
text_size	normal or large, which determines the required ratio.
wcag_level	Which standard (AA or AAA) was applied.
result	PASS or FAIL.

The summary file has one row per page, showing total checks, pass/fail counts, the fail rate as a percentage, the lowest and highest contrast ratios found, and an overall page result (PASS, FAIL, SKIP, ERROR, or NO_DATA if no checkable text was found).

9. A Complete Walkthrough

Here is what a full audit of a fictional website, <https://example.gov.in>, looks like from start to finish, run from inside the folder containing the four scripts.

Step 1 — Crawl the site to build the master page list

```
python crawler.py --url https://example.gov.in --output crawl.csv
```

This produces `crawl.csv`, listing every page the crawler could reach.

Step 2 — Inventory all media on the site

```
python media_crawler.py --input crawl.csv
```

This produces `crawl_media_detail.csv` and `crawl_media_summary.csv`.

Step 3 — Check every image for proper alt text

```
python alt_text.py crawl.csv wcag_alt_report.csv
```

This produces `wcag_alt_report.csv`.

Step 4 — Check every page for adequate colour contrast

```
python contrast_checker.py crawl.csv --level AA
```

This produces two timestamped files, e.g. `contrast_detail_20260617_143000.csv` and `contrast_summary_20260617_143000.csv`.

At the end of these four commands, you have everything needed for an accessibility audit report: a full sitemap, a media inventory, an alt-text compliance report, and a colour contrast compliance report — all as spreadsheets ready to open in Excel.

10. Reading and Using the Reports

All four tools produce plain CSV files, which open directly in Microsoft Excel (just double-click, or use File > Open). A few general tips:

- Use Excel's filter / sort feature on the Result or page_result column to quickly group everything that needs attention.
- FAIL rows are candidates for the audit report, but should be opened and visually confirmed by a tester — the tools point you to the right page and element, but a human should make the final call.
- ERROR or SKIP rows usually mean the page could not be reached when the tool ran (it may have been temporarily down, very slow, or behind a login). These are worth re-checking manually or re-running the tool later.
- Because the Media Crawler, Alt-Text Checker, and Colour Contrast Checker all read from the same crawl CSV, their reports can be cross-referenced by page URL in Excel (e.g. with VLOOKUP) to build a single combined view of a page's accessibility status.

11. Important Limitations to Know

These tools are a major time-saver, but like any automated check, they can only see what they are designed to look at. Knowing their boundaries helps avoid both false confidence and wasted re-checking.

The Colour Contrast Checker mainly reads inline styling

It looks at colours set directly on an element (an inline style="color:..." attribute) or inherited from a parent element's inline style. Many modern government websites set their colours through external stylesheet files instead. On those pages, the checker falls back to a default assumption of black text on a white background, which may not reflect what a visitor actually sees. Pages with very few PASS/FAIL rows, or where every result looks identical, are a sign the page's real styling lives in an external stylesheet and deserves a manual look in a browser.

All three checking tools only see what loads without logging in

Pages behind a login, a CAPTCHA, or built entirely with JavaScript that loads content after the page first appears may not be fully visible to these tools, the same way they would not be fully visible to a simple, fast scan. Such pages are good candidates for manual review.

A PASS result is not a guarantee of full compliance

WCAG and GIGW 3.0 cover many more rules than alt text and colour contrast — keyboard navigation, heading structure, form labelling, and more — which this toolkit does not check. These tools cover two of the most common and impactful checks, not the entire guideline set.

12. Troubleshooting

Problem	Likely cause and fix
"ModuleNotFoundError" or "import error" when running a script	A required helper package is missing. Re-run the setup command in Section 4.2: <code>pip install requests beautifulsoup4 pandas openpyxl tinycss2</code>
The crawl seems to run forever	The site is very large, or the crawler has found an effectively endless set of pages (e.g. a calendar that links forward indefinitely). Re-run with a depth limit, e.g. <code>--depth 4</code> , to keep it bounded.
Lots of TIMEOUT or connection errors	The website may be slow or temporarily overloaded. Try increasing <code>--timeout</code> and <code>--delay</code> so the tool waits longer and is gentler on the server.
SSL / certificate errors	Common on internal or staging government sites with self-signed certificates. The Colour Contrast Checker supports <code>--no-verify</code> to bypass certificate checking for these cases.
The script ends with an error-like exit but no crash message	This is by design. <code>alt_text.py</code> and <code>contrast_checker.py</code> deliberately end with a non-zero exit status when issues are found, so the tools can be plugged into automated checks. Look at the report file itself — the run still completed and saved its data.
The media or contrast report has very few rows for a page that clearly has lots of content	See Section 11 — this usually means the page's styling or media references live in an external stylesheet/JavaScript file rather than inline HTML, which these tools read less completely.

13. Quick Command Reference

A cheat-sheet of the core command for each tool, in pipeline order.

```
# Step 1 – crawl the site
python crawler.py --url https://example.gov.in --output crawl.csv

# Step 2 – inventory all media
python media_crawler.py --input crawl.csv

# Step 3 – check image alt text (WCAG 1.1.1)
python alt_text.py crawl.csv wcag_alt_report.csv

# Step 4 – check colour contrast (WCAG 1.4.3 / 1.4.6)
python contrast_checker.py crawl.csv --level AA
```

14. Glossary

Term	Plain-language meaning
WCAG	Web Content Accessibility Guidelines — the international standard this toolkit's checks are based on.
GIGW 3.0	Guidelines for Indian Government Websites, version 3.0 — India's government standard, which builds on WCAG.
Alt text	A short written description attached to an image so that screen readers and browsers can describe it when the image itself can't be seen.
Contrast ratio	A number describing how clearly text stands out from its background colour; higher is more readable.
AA / AAA	Two levels of the WCAG contrast rule. AA is the everyday standard; AAA is the stricter, optional standard.
Crawl / crawl depth	Crawling means automatically following links from page to page across a website. Depth is how many links away from the homepage a page is.
CSV	A simple spreadsheet file format (Comma-Separated Values) that opens directly in Excel.
HTTP Status	A standard code a web server returns for each page request, e.g. 200 (success), 404 (not found), 500 (server error).
Screen reader	Software used by people who are blind or have low vision that reads webpage content aloud or converts it to braille.
Inline style / external stylesheet	Inline styling is formatting written directly on an HTML element; an external stylesheet is a separate file of formatting rules applied to many pages at once.

End of manual.